

E06 · 무엇이 sleep-time compute 인가 — 이름과 실제

이 권은 본문 ch11(판별식과 전수 분류) 과 ch24(무엇이 실제로 이범주인가) 를 수식과 증명 없이 다시 설명한다. 답할 질문은 둘이다. 어떤 절차가 sleep-time compute 이고 어떤 절차가 아닌가, 그리고 이름과 실제가 어긋나는 것은 우연인가 구조인가.

앞 권들은 부품을 놓았다 — 학습이 무엇인지, 데이터가 어디서 오는지, 왜 오프라인이 필요한지. 이 권은 처음으로 **자를 든다**. 그리고 그 자를 문헌 29 편 전체에 대 본 결과가 이 책의 첫 번째 실질 발견이다. 결론을 미리 적는다. **이름이 기제보다 넓다**. 제목에 consolidation 이 들어있는데 가중치를 한 번도 건드리지 않는 논문이 있고, 배포하고 나면 아무것도 바뀌지 않는 논문이 이 이름 아래 인용된다.

1. 이름표로는 가를 수 없다

독자의 세계에도 같은 병이 있다. “cache” 라는 낱말 하나가 prefix cache, KV cache, CDN cache, page cache 를 전부 가리키고, 넷은 무효화 규칙도 상주위치도 복구 절차도 다르다. 그런데도 회의에서 “캐시 때문입니다” 라는 문장은 통한다 — 각자 다른 것을 떠올린 채로.

이 라인은 그 상태가 훨씬 심하다. 세 편의 논문이 자기 절차를 전부 sleep 이라 부르는데, 하나는 **데이터베이스의 행**을 고치고, 하나는 **세션에 딸린 상태**를 고치고, 하나는 **배포되는 가중치 파일**을 고친다. 셋의 롤백 절차는 각각 레코드 삭제, 세션 종료, 체크포인트 되돌리기다. 같은 낱말 하나가 이 셋을 덮고 있다.

기호 풀이 Θ (대문자 세타, slow weights) = 배포되는 가중치 그 자체. `.safetensors` 로 굽는 것. W (fast weights) = 질의를 처리하는 동안 값이 움직이는 고정 크기 상태. 세션이 끝나면 사라진다. E (external store) = 모델 밖의 텍스트 · 벡터 · 그래프 저장소. 당신의 어휘로는 vector DB 한 벌 또는 사용자별 메모 테이블. q = 사용자 질의 하나. t = 토큰 첨자, k = **sleep 라운드** 첨자 (오프라인 정리작업의 회차), τ = 세션 첨자. 이 셋은 서로 다른 시계이며, 이 라인에서 가장 자주 일어나는 서술 오류가 셋을 뒤섞는 것이다.

그래서 판정의 입력에서 **논문이 자기를 무엇이라 부르는가**를 통째로 빼야 한다. 제목도, 초록도, 저자의 요약도 보지 않는다. 보는 것은 두가지뿐이다 — **무엇을 바꾸는 식인가**, 그리고 **그 식이 언제되는가**.

비유 판별식은 세관 검사다. 여행자가 신고서에 무엇이라 적었는지가 아니라 가방을 열어 무엇이 들었는지로 판정한다. — **비유가 깨지는 곳**: 세관은 물건만 보면 되지만 판별식은 **시각표까지** 본다. 같은 물건이라도 도착 전에 들어왔는지 도착 후에 들어왔는지가 판정을 뒤집는다. 그리고 세관은 통과 못한 물건을 압수하지만, 여기서 탈락한 논문은 버려지지 않고 다른 자리로 옮겨져 증거가 된다 (§ 11).

자세한 것은 ch11 § 11.1.

2. 판별식 — 네조건

판별식. 어떤 절차가 sleep-time compute 인가? 네 가지를 묻는다. (1) **시계** — 상태를 바꾸는 그 연산이 질의 q 가 도착하기전에 도는가. (2) **예산** — 그 연산이 추가 계산을실제로 태우는가. (3) **대상** — 그 연산이 $\Theta \cdot W \cdot E$ 중 어느 한 층의 값을 실제로 바꾸는가. (4) **소비자** — 바뀐 상태를 다음에오는 다른 질의가 쓰는가. 넷을 모두 만족해야 한다. 하나라도빠지면 인접 문헌이지 이 범주가 아니다.

조건 (2) 에 쓰는 값에 기호를 준다.

기호 풀이 $B_s = \text{sleep 예산}$. 질의가 오기 전에 도는 오프라인 연산 한번이 태우는 계산량이며, 단위는 FLOPs 또는 토큰 수다. 조건 (2) 는 $B_s > 0$ 을 요구한다 — 즉 **0 보다 크기만 하면** 된다. 얼마나큰지는 묻지 않는다. $L_w = \text{wake 지연 기여}$. 사용자가 기다리는 시간에 그 절차가 더하는 밀리초 (TTFT·ITL). 조건 (1) 을만족하는 절차는 정의상 L_w 에 0 을 기여한다.

네 조건은 서로 독립이 아니라 **직교**한다. (1) 은 시간축을, (2) 는비용축을, (3) 은 상태축을, (4) 는 재사용축을 각각 **한 번씩만**자른다. 그래서 넷 중 어느 하나를 빼면 그 축이 통째로 열리고, 열린 축을따라 **매번 다른 종류의 논문**이 잘못 들어온다.

판별식이 **묻지 않는 것** 들을 못 박아 둔다. 첫째, **논문의자기 규정**. 제목에 sleep 이 있는지, 저자가 자기 절차를 consolidation 이라부르는지는 판정에 들어오지 않는다. 둘째, **성능**. 이득이 있는지없는지는 이 범주에 속하는지와 무관하다. 판별식은 **소속**을 정하는도구이지 **가치**를 정하는 도구가 아니다. 어느 쪽이 나온가는 뒤의권이 답한다.

자세한 것은 ch11 § 11.2.1.

3. 조건 하나를 빼면 무엇이들어오는가

표 E06-1. 네 조건이 각각 자르는 축과, 그 조건을 뺐을 때 잘못 들어오는논문.

조건	자르는 축	빼면 들어오는 것	대표 사례
(1) 시계	시간	사용자 턴 안에서 하는 모든 쓰기	MemGPT (arXiv:2310.08560)
(2) 예산	비용	추론 없는 순수 저장 · 색인	RAG (arXiv:2005.11401), Memorizing Transformers (arXiv:2203.08913)
(3) 대상	상태	읽기만 바꾼 논문	Memory Caching (arXiv:2602.24281)
(4) 소비자	재사용	그 질의 안에서 소비되고 끝나는 계산	chain-of-thought, Nested Learning (arXiv:2512.24695)

(1) 을 빼면 모든 쓰기가 **sleep** 이 된다. MemGPT 는 대화 도중에 함수호출을 디코딩해서 external store 를 고친다. 그 연산은 답을 만드는 것과같은 **예산**에서 나가고, 실행 오류

가 나면 그 자리에서 되먹여진다 [MemGPT § 2.3]. 조건 (1) 이 없으면 이런 쓰기와 밤새 도는 sleep 라운드의 쓰기가 구분되지 않고, “sleep-time compute” 은 그냥 “쓰기” 의 다른 이름이 된다.

시스템 모델링 관점 조건 (1) 이 서빙에서 뜻하는 것은 명확하다. **비용이 어느 쪽예산에서 나가는가.** 턴 안에서 쓰는 절차는 그 쓰기 시간이 통째로 L_w 에 얹힌다 — 요약 한 번을 만들려고 수백 토큰을 decode 하면 그것은 곧 사용자가 보는 TTFT 이고, p99 SLA 를 잡아먹는 항이며, 배치 스케줄러입장에서는 **decode 슬롯을 점유하는 또 하나의 요청**이다. 질의 전에도는 절차는 그 항이 0 이다. 대신 비용이 사라진 것이 아니라 **다른회계 장부로 옮겨 간다** — 유휴 시간의 GPU 점유, 즉 오프피크 용량이다. 두 절차가 태우는 FLOPs 가 똑같아도 하나는 SLA 문제이고 하나는 용량 계획문제다. 판별식의 첫 조건은 문헌 분류 규칙이기 이전에 **이 두장부를 가르는 선**이다.

(2) 를 빼면 **저장이 추론으로 승격된다**. RAG 는 문서를 잘라 인코딩해색인에 넣을 뿐, 넣기 전에 아무 추론도 하지 않는다. Memorizing Transformers 도 같은 자리에 있다 — 저장되는 것이 그 층의 forward pass 가 **이미 만든** (k, v) — attention 의 key·value 텐서 — 그대로이고, 논문 스스로 kNN 조회가 먼 토큰을 평균하거나 요약하지 않고 정확한 값을 회수한다고 쓴다 [Memorizing Transformers § 1]. 조건 (2) 가 없으면 색인 구축이 전부 sleep-time compute 이 된다.

비유 조건 (2) 는 **복사기와 요약자**를 가른다. 복사기는 원본을 그대로 한장 더 만들고, 요약자는 읽고 생각한 뒤 새 문서를 만든다. 판별식은 요약자만 통과시킨다. — **비유가 깨지는 곳**: 복사기와 요약자는 사람이 보면 구분되지만, 파이프라인 다이어그램에서는 둘 다 “전처리단계” 라는 같은 네모로 그려진다. 실제로 무엇이 도는지는 그 네모 안에 **모델 호출이 있는가**를 봐야 알 수 있고, 그것이 이 조건을 기계적으로 판정하는 방법이다.

(3) 을 빼면 읽기를 바꾼 논문이 쓰기를 바꾼 논문으로 둔갑한다. Memory Caching 이 그 경우다 (§ 7.2). (4) 를 빼면 **test-time compute 전체가 들어온다**. chain-of-thought 도 상태를 만들고 그 상태를 읽지만, 만든 질의가 끝나면 사라진다. 이 라인을 만든 논문이 자기를 **test-time 예산의 대안**으로 제시했으므로, 이 조건은 특히 양보할 수 없다.

자세한 것은 ch11 § 11.2.3.

4. 다섯 단계로 기계적으로 판정하기

판정에 재량이 끼지 않게 하려면 순서가 고정되어야 한다. 본문은 다섯 단계를 쓴다.

적용 절차. 1 단계. 논문의 수식 중 **상태를 좌변에, 이전 상태를 우변에** 놓은 것을 전부 뽑는다. 하나도 없으면 조건 (3) 에서 탈락이다. **2 단계.** 그 좌변이 어느 층인지 배정한다 — 배포아티팩트의 일부이면 Θ , 요청·세션에 붙는 가중치형태의 상태이면 W , 모델 밖 레코드이면 E 다. **3 단계.** 그 식의 **첨자를 읽는다**. 토큰이면 t , sleep 라운드면 k , 세션이면 τ 다. 첨자가 답을 만드는 중의 토큰이면 조건 (1) 에서 탈락이다. **4 단계.** 그 식이 **추가 추론을 태우는가**. 저장·복사·색인만 하면 $B_s = 0$ 이고 조건 (2) 에서 탈락이다. **5 단계.** 바뀐 상태를 읽는 질의가 그것을 만든 질의와 **다른가**. 같으면 조건 (4) 에서 탈락이다.

이 절차의 핵심은 **입력이 무엇인가**에 있다. 1-2 단계는 논문의수식만 보고, 3-5 단계는 논문의 실험 설정만 본다. 저자의 서술은 **어느 단계에도 들어가지 않는다**. 그래서 판정이 논문의 자기 규정과어긋나는 일이 실제로 생기고, 어긋나는 자리에서는 이 책이 자기 판정을따른다.

실무자용으로 줄이면 세 줄이다. 수식의 **왼쪽**을 보고 어느 층인지정하고, **첨자**를 보고 언제인지 정하고, 그 사이에 **모델호출**이 있는지 본다. 제목은 마지막까지 보지 않는다.

자세한 것은 ch11 § 11.2.2.

5. 판별식이 판정하지 못하는 두자리

도구의 한계를 먼저 밝힌다. 회색지대가 둘 있고 둘 다 의도된 것이다.

첫째, 시계를 스스로 정하지 않는 논문. LoRA(arXiv:2106.09685)·ROME(arXiv:2202.05262)·MEMIT(arXiv:2210.07229) 이여기 해당한다. 셋 다 “무엇을 어디에 어떻게 쓰는가” 는 완전히 규정하지만 “언제 쓰는가” 는 규정하지 않는다. 3 단계에서 읽을 첨자가 아예없으므로 판정이 성립하지 않는다. 이 책은 이들을 불충족으로 적되 탈락사유를 **시계 부재**로 따로 표기하고, 쓰기 **primitive** 라는별도의 지위를 준다.

비유 이 셋은 절차가 아니라 **연장**이다. 망치가 언제 쓰이는지는 망치가정하지 않는다. — **비유가 깨지는 곳**: 망치는 아무 때나 집어들어도 되지만 Θ 를 고치는 연장은 그렇지 않다. 한 번 쓰는 순간**배포되는 아티팩트가 바뀌므로**, 그 연장을 어느 시계에 없느냐가 곧롤백 단위·배칭 가능성·회귀 검증 주기를 통째로정한다. 연장 선택보다 시계 선택이 무거운 결정이다.

둘째, 조건 (2) 는 부등식이라 크기를 묻지 않는다. SCM(arXiv:2604.20943) 의 sleep 라운드는 LLM 호출 없이 그래프 연산만 돌고 [SCM Algorithm 1], LM Need Sleep(arXiv:2606.03979) 의 sleep 라운드는 distillation 과 RL 을 돈다. 두 예산은 자릿수가 전혀 다른데판별식은 둘에 **같은 판정**을 준다.

이 무차별은 결함이 아니라 역할 분담이다. 판별식은 **소속**을정하고, 크기는 통일 비용 회계가 정한다 (→ ch25). 판별식 하나로 “어느 쪽이비싼가” 에 답하려 하면 도구를 잘못 쓰는 것이다.

두 회색지대를 명시하는 것 자체가 판별식의 **반증 조건**이기도 하다. 네 조건을 모두 만족하는데 이 책이 배제한 절차가 발견되면 그것은 이 책의오류다.

자세한 것은 ch11 § 11.2.4.

6. 29 편을 전부 대 본결과

절차를 corpus 전체에 돌리면 이렇게 갈린다. 이 책의 모든 집계가 이 규약을쓴다.

표 E06-2. corpus 29 편의 갈림.

군	편수	무엇인가
통과	9	네 조건을 모두 만족한다

군	편수	무엇인가
경계 사례	1	MemGPT. 조건 (1) 만 불충족이며 통과군으로 세지 않는다 (§ 8)
불충족 — B 군	9	이 이름 아래 자주 함께 인용되지만 어느조건에선가 탈락한다
불충족 — C 군	10	애초에 이 범주를 자처하지 않는 배경 · 제약 문헌. 판정 대상이 아니다

통과 9 편을 층으로 읽으면 극단적으로 치우쳐 있다.
표 E06-3. 통과 9 편의 경로 분포와 시계.

경로	편수	논문	쓰기가 도는 주기
E	6	Letta Sleep-time Compute(2504.13171) · Mem0(2504.19413) · Zep(2501.13956) · Reasoning-Bank(2509.25140) · SCM(2604.20943) · Multi-Timescale(2605.05097)	문맥당 1 회 / 메시지쌍마다 / 에피소드마다 / 과제 종료 후/ 임계 도달 시 / 이벤트마다
Θ	2	SEAL(2506.10943) · LM Need Sleep(2606.03979)	두 시간척도/ sleep 라운드 k 마다
W	1	Do LMs Need Sleep?(2605.26099)	KV cache 를 비우기 전 정해진 횟수만큼 재귀

두 가지가 즉시 나온다. 첫째, W -경로는 **표본이 1** 이다. 경계사례를 더하면 E 쪽은 7 편이 된다. 어떤 판정이든 표본 1 인 경로에 대해서는 강하게 말할 수 없고, 그 사실을 밝히는 것이 유일한 정직한처리다. 둘째, 통과군 안에서도 “**질의 전**” 의 주기가 **논문마다 다르다** — 오른쪽 열이 그 이질성이며, 이것이 회계에 직접 얹힌다.

기호 풀이 N_q = 한 문맥을 공유하는 질의 수. 오프라인 비용을 나눠 갠때의 분모다. 한 번 미리 계산해 둔 것을 뒤에 오는 질의 몇 개가 나눠쓰는가를 센다.

N_q 가 “한 문맥당 질의 수” 인 논문과 “메시지쌍당 질의 수” 인 논문과 “과제당 질의 수” 인 논문의 값은 자릿수가 다르다. 그러므로 **통과군에 함께 속한다는 사실만으로 두 논문의 비용 논증을 나란히놓으면 안 된다.**

자세한 것은 ch11 § 11.3.

7. 이름과 실제가 갈리는 다섯사례

다섯 편을 고른 이유는 어긋남의 유형이 서로 다르기 때문이다.

7.1 SCM — 제목은 consolidation, 가중치는불변

Sleep-Consolidated Memory는 이름의 세 낱말 중 둘이 Θ -경로를 가리킨다. 절차를 밟으면 다른 답이 나온다. 갱신연산자는 다섯이고 (기존 edge 강화·전역 downscale·장기 저장소로 transfer·REM 단계의 dream-prune) [SCM Algorithm 1], 그 좌변은 전부 NetworkX 그래프와 텍스트 버퍼 — 즉 **모델 밖**이다 [SCM § 3.5]. 층은 E 다. 네 조건은 모두 만족하므로 **통과이되** 경로는 E 다. 논문 자신이 못 박는다 — 학습이나 fine-tuning 이 필요 없고 모든 구성요소가 기존 pre-trained 모델 아니면 알고리즘적 논리를 쓴다 [SCM § 3.8]. 가중치 delta 도, gradient 도, fast weight 상태도 논문 어디에도 없다.

원인은 제목이 아니라 **연산자의 이름을 층과 혼동한 것**이다. consolidation·downscaling·pruning 은 신경과학에서 시냅스에 걸리는 연산이고, 시냅스의 인공 대응물은 가중치다. SCM 은 같은 연산자를 **그래프의 edge** 에 건다. 연산자가 같아도 좌변이 다르면 다른 층이고, 비용·용량·롤백단위가 전부 달라진다.

7.2 Memory Caching — 예산이 0 이고, 바뀐 것은읽기다

이 논문은 recurrent 모델의 고정 용량이 recall 을 무너뜨린다는 진단에서 출발해, 과거 memory 상태의 체크포인트를 버리지 않고 들고 있다. 문제의식이 겹치므로 함께 거론되기 쉽다. 절차는 두 자리에서 걸린다. **갱신 규칙이 baseline 에서 한 글자도 바뀌지 않았고**, 바뀐 것은읽기다 — 단일 상태를 읽던 자리에서 캐시 집합 위의 게이트된 혼합을 읽는다 [Memory Caching Eq. 4, 7]. 상태를 바꾸는 연산이 아니라 **보존하고 고르는** 연산이므로 조건 (3) 에서 탈락한다. 그리고 그 보존은 이미 계산된 값을 남겨 두는 것이므로 $B_s = 0$, 조건 (2) 에서도 탈락한다.

비유 이 논문이 한 일은 문서를 다시 쓴 것이 아니라 **옛 판본을 버리지 않고 서가에 남겨 둔 것**이다. — **비유가 깨지는 곳**: 서가는 자리를 차지하고 사서가 관리해야 하지만, 여기서 보존되는 iterate 는 forward pass 가 어차피 만들어 낸 값이라 **만드는 데 드는 추가 비용이 정확히 0** 이다. 저장 비용만 있고 생산 비용이 없다는 것이 이 논문이 조건 (2) 에서 걸리는 이유 그 자체다.

탈락했다고 버려지지 않는다. 고정 크기 상태 하나로는 recall 이 무너진다는 것을 정량화하고, 그러면 몇 개를 들고 있어야 하는지를 차수로 답한다 — 로그 분할이면 캐시 개수가 $\log_2 L$ 이하다 [Memory Caching § 4.2]. L = 시퀀스 길이다. 이 답은 W 층 용량 천장의 재료로 다시 쓰인다.

7.3 Nested Learning — 연속체는 표기의다리다

이 논문은 갱신 주파수라는 **하나의 축** 위에 pre-training 부터 attention 까지를 늘어놓고, Θ 를 그 축의 아무것도 움직이지 않는 끝점으로 놓는다. 축이 하나이므로 W 와 Θ 사이에 다리가 놓인 것처럼 읽힌다.

3 단계에서 그 읽기가 무너진다. 논문의 모든 시계가 **살아 있는 시퀀스의 토큰**이다. 실험이 스윙하는 가장 느린 주파수조차 512 토큰과 2K 토큰이므로 [NL § 9.1], 긴 문서 하나 분

량이 세션도 sleep 라운드도 아니다. 5 단계도 걸린다 — 논문 스스로 획득된 in-context 지식은 현재 문맥이 제거되면 존속하지 않는다고 쓰고 [NL § 6], 해당 블록은 일정 step 뒤 초기값으로 되돌아간다 [NL § 7.1]. 결정적인 것은 논문 자신이 오프라인 단계를 **범위 밖에 둔다**는 사실이다. 신경과학의 두 단계를 소개한 뒤 이 연구는 첫 단계, 즉 온라인 과정에 집중한다고 선언한다 [NL § 1].

연속체는 표기의 다리이지 기제의 다리가 아니다. 하나의 축이 두 끝점을 이름 위에서 잇는 것과, 어떤 갱신이 실제로 문맥 경계를 넘어 체크포인트에 되쓰이는 것은 다른 사건이다. 이 논문에서 후자는 한 번도 일어나지 않는다.

7.4 Memory Layers — 배포하고 나면 정지한다

이 논문은 Θ 의 일부를 대역폭 지배 구간으로 떼어 내 10^2 배키운다. 최대 설정이 6400 만 key, 즉 1,280 억 개의 memory 파라미터이고 [Memory Layers § 5.2], 그만큼 늘려도 FLOPs는 사실상 불변이다 [Memory Layers § 5.1]. 그런데 판별식은 통과하지 못한다. Θ 가 움직이는 **유일한 시점이 pre-training**이고, 배포 후에는 $\Theta \cdot W \cdot E$ 어느 것도 바뀌지 않는다. 3 단계에서 볼 시계가 없어 (1)에서, 배포 후 상태 변화가 없어 (3)에서 탈락한다. 논문이 continual learning과 망각 감소를 거론하는 대목은 결론의 희망 서술 한 문단이며 실험이 아니다 [Memory Layers § 6].

시스템 모델링 관점 이 논문이 서빙에 던지는 것은 예외적인 성질 하나다. Θ -경로는 일반적으로 **사용자별 delta**를 만들고, delta가 생기는 순간 한 별의 가중치를 여러 요청이 함께 쓰는 shared-weight batching이 깨진다. 그런데 여기서 늘어난 1,280 억 파라미터는 **전 사용자 공유**다. 배치가 그대로 성립하고, 늘어난 것은 파라미터 수이지 토큰당 FLOPs가 아니므로 병목은 연산이 아니라 **HBM 대역폭과 용량**으로 옮겨 간다 — decode 시간의 지배항이 곧 “이 거대한 테이블에서 몇 개의 슬롯을 긁어 오는가”가 된다. **배포 후 정지가 대가이자 동시에 이점이다.** 개인화를 포기한 대신 배치를 지켰다. 판별식이 이 논문을 범주 밖으로 보내는 것과, 이 논문이 개인화 문제를 아예 갖지 않는 것은 같은 사실의 두 얼굴이다.

7.5 Generative Adapter — 산출물의 모양과 시계가 갈린다

이 논문은 문맥을 받아 **한 번의 forward로 adapter를 만들어 낸다**. 산출물의 모양이 LoRA이고 논문이 그 형태를 명시적으로 유도한다 [Generative Adapter Eq. 8–9]. 모양만 보면 전형적인 Θ -경로 아티팩트다. 그러나 base 가중치는 미분되지도 수정되지도 않는다 — 이 절차이후의 체크포인트가 이전과 **바이트 단위로 같다**. 배포 아티팩트가 바뀌지 않으므로 Θ -경로가 아니다. 그리고 시계가 wake다. 문맥이 주어지면 **그 자리에서** 접고 그 결과로 답한다 [Generative Adapter § 4.1]. 질의 도착 전에 도는 pass가 아니므로 조건 (1)에서 탈락한다.

다섯 사례가 함께 말하는 것은 하나다. **분류는 산출물의 모양이 아니라 (갱신식, 시계)의 쌍으로 한다.** 모양으로 분류하면 Generative Adapter가 Θ -경로로, SCM이 Θ -경로로, Memory Caching이 sleep-time compute으로 들어온다. 셋 다 틀린다.

자세한 것은 ch11 § 11.4.

8. 경계 사례 — MemGPT는 되고 RAG는 안 되는이유

MemGPT는 조건 (2)·(3)·(4)를 만족하고 (1)만 만족하지 않는다. 그런데 그 쓰기의 산출물은 이후 질의에 쓰이고, E -경로의 통과 논문들이 자기 계보를 여기서 시작한다. 조상을 빼면 뒤 논문들의 문제의식이 허공에서 시작하고, 조상을 통과로 세면 조건 (1)이 무력해진다. 그래서 규칙이 필요하다.

경계 사례 규칙. 네 조건 중 셋을 만족하고, 통과군의 논문이 그 논문을 자기 계보의 조상으로 지목하는 경우에 한해, 해당 경로의 출발점으로 서술한다. 단 (i) 탈락한 조건이 무엇인지 본문에 명시하고, (ii) 통과군의 일원으로 세지 않는다.

RAG는 반대 방향의 예다. 조건 (2)에서 탈락하고, 통과군 논문이 조상으로 지목하지도 않는다 — 이 라인의 이름을 만든 논문의 참고문헌 22개에 RAG가 없다 [Letta STC 참고문헌]. 따라서 RAG는 경계 사례가 아니라 E 층의 형태와 읽기 비용을 정의한 배경이다. 경계 사례 자격은 탈락 조건의 개수만으로 주어지지 않는다 — 계보적 조상과 범주 소속은 다른 문제이고, 규칙은 그 둘을 분리한다.

자세한 것은 ch11 § 11.5.

9. 이름이 기제보다 넓다 — 그리고 그것은 우연이 아니다

여기까지가 증거다. 이제 판정이다. 어긋남이 단순한 정리 부족이라면 분포가 무작위여야 한다 — 오래된 논문일수록 흔들리고 최근일수록 수렴해야 한다. 실제 분포는 그렇지 않다. 어긋남은 경로를 따라 정렬되어 있고, 정렬의 축은 기술이 아니라 공동체다.

표 E06-4. 같은 이름을 쓰는 세 공동체.

	E -경로	W -경로	Θ -경로
대화 상대	에이전트 기억	test-time learning	지속학습
물려받은 서식	시스템·제품 논문	아키텍처 논문	학습 논문
비용을 적는 칸	지연·정확도 (쓰기 비용 칸 없음)	상태 크기·throughput	step·batch·GPU-hour
대조군의 관례	경쟁 제품, full-context	같은 폭의 아키텍처 변형	사전학습 체크포인트
이 이름을 쓰는가	창시 논문만, 후속은 쓰지 않음	제목에 쓰되 정의는 자기 것	제목에 쓰되 정의는 자기 것

W -경로 열은 통과군 표본이 1편이므로 관행이 아니라 그 한 편의 서술이다.

증거 셋이 같은 방향을 가리킨다.

첫째, 보고 관행. B_s 를 무엇으로든 보고한 비율의 경로 간순서는 $\Theta > E$ 다. 원인은 기제가 아니다 — Θ -경로가 계산을 더 태워서가 아니라, 그쪽 논문이 학습 논문의 서식을 물려받아 step·batch·GPU-hour 칸을 애초에 갖고 있기 때문이다. E -경로가 물려받은 서식에는 쓰기 비용을 적는 칸이 없다. 같은 연산을 하는 두 논문이 다른 칸을 채우는 것은 기술적 선택이 아니라 서식의 상속이다.

둘째, 침묵의 방향. 침묵이 연대 때문이라면 뒤에 온 논문이 앞선 논문을 인용해야 한다. 실제로는 반대 방향의 공백이 반복된다. 이 이름을 만든 논문은 8개월 뒤 다른 경로 논문에

게 인용되지 않고, 5 개월 뒤같은 경로의 후속 논문에게도 인용되지 않으며, 그 후속 논문에는 “sleep” 이라는 단어가 한 번도 나오지 않는다. 한 건이면 인용누락이지만 반복되면 검색어의 부재다.

셋째, 이름의 이동 경로. 이 라인의 이름을 만든 논문에서 sleep 은 consolidation 이론의 용어가 아니라 “상호작용 사이에 모델이 노는 시간” 을가리키는 조작적 낱말이고, 참고문헌 22 개에 생물학 문헌이 0 편이다 [Letta STC § 1]. **이름은 생물학에서 왔고 논증은 오지 않았다.** 그러므로같은 이름을 쓰는 세 논문이 같은 가설의 변형이라고 읽을 근거가 없다.

비유 세 공동체는 환승역이 없는 세 노선이다. 각자 자기 종점까지 잘 달리지만서로의 승객을 받지 못한다. — **비유가 깨지는 곳:** 지하철은노선도가 있어 갈아탈 자리가 어디인지 최소한 **보이기라도** 한다. 여기에는 그 노선도에 해당하는 것이 없다 — 세 경로가 공유하는벤치마크의 교집합이 공집합이고, 상태 용량을 바이트로 적은 논문이 corpus 에 0 편이며, 예산을 절대 단위로 적은 논문이 1 편이다. **서로를읽기 시작하더라도 나란히 놓을 축이 없다.**

그러므로 이 간격은 시간이 해결하는 종류가 아니라 **누군가 축을세워야** 닫히는 종류다.

여기에 독립 확증이 하나 붙는다. 판별식의 분류는 갱신식과 시계를 읽어서나왔고, B_s 보고 감사는 비용 칸 하나만 보고 “질의 전 오프라인 pass 가있는가” 를 판정했다. 두 절차는 입력이 다르고 서로를 참조하지 않는데**결과 집합이 사실상 겹친다** — 같은 9 편이 같은 자리에 떨어진다. 범주 밖이라는 판정과, 그 논문에는 sleep 예산이라는 칸 **자체가없다**는 관측은 같은 사실의 두 얼굴이다.

자세한 것은 ch24 § 24.2 와 § 24.3.1.

10. 간격을 방치하면 무엇이틀리는가

셋이 틀린다. 셋 다 문헌의 결함이 아니라 **문헌을 읽는 쪽의결함**이며, 그래서 이 절이 실무에 가장 직접 닿는다.

잘못된 비교. 같은 이름 아래 놓인 두 수치가 같은 축의 값이아니다. 이 이름을 만든 논문의 $5\times$ 는 **생성 토큰 축**의 수평 이동이고 prefill 항을 세지 않는다. Θ -경로가 보고하는 예산은 optimizer step 이고, 그것을 FLOPs 로 옮기려면 MFU 나 달성 throughput 이 필요한데그것을 보고한 논문이 이 corpus 29 편 중 1 편이다. 읽기식을 바꾼 논문의비용은 애초에 상각 대상이 아니어서 분모 N_q 를 키워도 줄지 않는다. **세 값은 단위가 다른 정도가 아니라 회계의 자리 자체가 다르다.** 그런데 이름이 같으므로 나란히 놓인다.

이 실패가 가장 선명한 자리가 상각의 분모다. N_q 의 분포를 알아야상각을 논할 수 있는데 **상각을 주장하는 논문 중 그 분포를 인쇄한것이 없다.** 정작 그 분포를 실은 논문은 판별식에서 탈락해 **상각할것이 애초에 없는** 쪽이다. 상각을 주장하는 쪽에는 분모가 없고, 분모를 가진 쪽에는 상각할 것이 없다.

잘못된 기대. 이름이 consolidation 을 약속하므로 독자는 “sleep 라운드를 늘릴수록 좋아진다” 를 기대한다. 문헌은 그 기대를 지지하지않는다. 이 이름을 만든 논문에서 sleep 스케일링은 **비단조**이고 —병렬 5 벌이 대체로 10 벌보다 낮다 — 이득은 높은 test-time 예산에서 두 과제모두 **역전한다** [Letta STC § 5.1, § 5.2]. E -경로에서는 논문자기 표의 품질 1 위가 반복적으로 **기억 시스템 없는 구성**이다 —full-context 72.90%가 Mem0 66.88%를 이긴다 [Mem0 Table 2]. 그리고 “늘릴수록 좋아진다” 보다 위험한 기대가 하나 더있다 —손실 곡선이 **평평하니 안전하다.** 평평한 1 차 지표 아래에서무너지는 좌표가 따로 있다.

잘못된 투자 판단. 세 경로는 서빙 아티팩트가 다르고, 그 차이가 인프라 결정을 통째로 가른다.

시스템 모델링 관점 “sleep-time compute에 투자한다”는 문장은 서로 다른 조달 결정 셋을 하나로 뭉친다. E -경로를 고르면 사용자별로 움직이는 파라미터가 없으므로 shared-weight batching이 온전하고, 상태는 데이터베이스 행과프롬프트 토큰에 산다 — 조달 대상은 **벡터 데이터베이스와 검색지연 예산**이다. Θ -경로를 고르면 사용자별 delta가 생기는 순간 그배칭이 깨지고, 기회의 성격이 HBM 대역폭이 아니라 **요청 지연 안에 delta를 끌어올 수 있는 웹 계층과 그 계층의 쓰기 내구성**으로 옮겨 간다. 같은 Θ -경로 안에서도 갈림길이 하나 더 있다 — 직접 편집 방식은 delta 아티팩트를 만들지 않고 공유 가중치를 고치므로 사용자별 개인화가 **구조적으로 불가능**하고, 하려면 모델 전체 사본이 필요하다. W -경로를 고르면 상태가 **활성 세션당**으로 붙으므로 동시 세션 수가 곧 상주 메모리이고, 조달 대상은 세션당 HBM 예산이다. 벡터데이터베이스인지, 웹 스토리지 계층인지, 세션당 상주 메모리인지가 그 문장안에서 결정되지 않는다.

세 실패의 공통 구조는 하나다. 이름은 **무엇을 하는가**를 가리키는데, 비교·기대·투자는 전부 **무엇이 바뀌는가**에 달려 있다. 판별식이 하는 일이 정확히 그 전환이다. 실무자의 어휘로 옮기면 네 질문이 된다.

어느 층이 바뀌는가 — 답이 E 이면 롤백은 행 삭제이고, Θ 이면 delta 인지 직접 편집인지를 다음으로 물으며, W 이면 동시세션 수가 곧 상주 메모리다. **시계는 언제인가** — 질의 전이면 L_w 에 0을 기여하고, 턴 안이면 그 비용은 사용자가 기다린다. **예산은 얼마인가** — 이 칸이 비어 있으면 상각 논증을 검증할 수 없으므로 그 방법에 대해 “효율적”이라고 쓰지 않는다. **누가 소비하는가** — 그 질의 자신이면 test-time compute이다.

네 질문 모두 **논문의 제목**으로는 답할 수 없다.

자세한 것은 ch24 § 24.3.2.

11. 이 판정을 무너뜨리는 관측

판정은 반증 조건과 함께 적어야 판정이다. 셋만 옮긴다.

분류가 틀렸음을 보이는 것. 네 조건을 모두 만족하는데 이 책이 불충족으로 넣은 절차가 있으면 해당 행을 고쳐야 한다. 가장 가까운 후보가 이미 지목되어 있다 — Memory Caching이 사후 학습 변형을 논하는 대목이며, 그 변형에 예산과 수치가 붙고 질의 전에 도는 것이 확인되면 그 변형에 대해서는 이 책의 탈락 판정이 성립하지 않는다.

조건 하나가 잉여임을 보이는 것. 넷 중 하나를 빼도 분류 결과가 하나도 바뀌지 않으면 그 조건은 잉여다. 현재는 어느 조건을 빼도 **매번 다른 논문**이 잘못 들어오므로 넷이 모두 일한다 (→ § 3).

간격이 구조적이지 않음을 보이는 것. 어느 경로 논문이 다른경로의 방법을 **baseline**으로 실행하거나 (한 문장짜리 기각이 아니라 같은 표 안의 수치여야 한다), 세 경로가 공유하는 벤치마크의 교집합이 생기거나, B_s 를 절대 단위로 적는 논문이 다수가 되면 — 간격은 구조가 아니라 시간이 해결할 지체였다는 뜻이다. **이 책은 그 일이 일어나기를 요구하는 쪽에 서 있다.**

반증되지 않는 것도 못 박는다. 개별 논문의 성능 수치가 재현실패로 내려가거나 후속 연구로 올라가도 이 권의 분류는 바뀌지 않는다. 판별식은 성능을 묻지 않기 때문이다.

자세한 것은 ch24 § 24.5.

요약

- 판별식은 네 조건이다 — (1) 질의 전, (2) 추가 계산을 태움, (3) 어느 한 층의 값을 실제로 바꿈, (4) 이후의 다른 질의가 소비. 넷은시간·비용·상태·재사용 축을 각각한 번씩 자르며, 어느 하나를 빼면 매번 다른 종류의 논문이 잘못들어온다.
- 판별식은 논문의 자기 규정도 성능도 묻지 않는다. 입력은갱신식과 시계뿐이고, 제목은 마지막까지 보지 않는다.
- 판별식이 판정하지 못하는 자리가 둘 있다 — 시계를 스스로 정하지 않는쓰기 primitive(LoRA·ROME·MEMIT), 그리고 예산의 크기. 후자는 비용 회계의 일이다.
- 전수 결과는 29 편 = 통과 9 + 경계 사례 1 + 불충족 19 다. 통과 9 편은 $E6/\Theta2/W1$ 로극단적으로 치우쳐 있고, W -경로는 표본이 1 이다.
- 이름이 기제보다 넓다. 제목의 두 낱말이 Θ 를가리키는데 가중치를 한 번도 건드리지 않는 논문이 있고 (SCM), 이 이름아래 인용되지만 배포 후 아무것도 바뀌지 않는 논문이 있다 (Memory Layers).
- 어긋남은 통과·불통과 경계에서만이 아니라 경로배정에서도 생긴다. 분류는 산출물의 모양이 아니라 (갱신식, 시계) 의 쌍으로 한다 — 모양으로 나누면 Generative Adapter 가 Θ -경로로, Memory Caching 이 이 범주로 잘못 들어온다.
- 경계 사례 규칙은 계보적 조상과 범주 소속을 분리한다. MemGPT 는조건 (1) 만 불충족이고 통과군이 조상으로 지목하므로 출발점으로 서술하되세지 않으며, RAG 는 조상 지목이 없으므로 경계 사례가 아니다.
- 간격은 구조적이다. 세 공동체가 각자의 서식과 어휘를 유지한 채같은 단어를 쓴다. 가장 강한 증거는 보고 관행이며, E -경로가물려받은 서식에는 쓰기 비용을 적는 칸이 없다.
- 이름은 생물학에서 왔고 논증은 오지 않았다. 같은 이름을 쓰는세 논문이 같은 가설의 변형이라고 읽을 근거가 없다.
- 간격을 방치하면 셋이 틀린다 — 회계의 자리가 다른 수치를 나란히 놓는비교, 늘릴수록 좋아진다는 기대, 서로 다른 조달결정을 하나로 뭉개는 투자 판단.
- 탈락은 폐기가 아니다. 분류의 목적은 배제가 아니라 각 문헌을자기가 실제로 증거가 되는 자리에 놓는 것이다.

자가 점검

- 판별식 네 조건을 순서대로 말하고, 각 조건을 뺐을 때 잘못 들어오는 논문을 하나씩 딸 수 있다.
- 임의의 논문에 다섯 단계를 밟아 통과·불통과를 판정하고, 불통과이면 걸린 조건 번호를 지목할 수 있다.
- SCM 이 왜 통과이면서 E -경로인지, Memory Layers 가 왜 파라미터를 1,280 억 개 늘리고도 이 범주가 아닌지를 각각 한 문장으로 말할 수 있다.
- “산출물이 LoRA 모양이면 Θ -경로다” 가 왜 틀린 판정 규칙인지 Generative Adapter 로 설명할 수 있다.
- MemGPT 는 왜 경계 사례이고 RAG 는 왜 아닌지, 탈락 조건 개수가 아닌 다른기준으로 설명할 수 있다.

- 통과 9 편의 경로 분포가 $E 6 / \Theta 2 / W 1$ 이라는 사실이 “어느 경로가 나온가” 라는 질문에무엇을 뜻하는지 한 문장으로 옮길 수 있다.
- (Rosetta) 어떤 절차가 L_w 를 늘린다면 왜 조건 (1) 에서 탈락하는지를, 자기 서빙 스택의 TTFT·p99 항목으로 설명할 수 있다.

다음 권으로

이 권은 대상을 확정했고 질문의 형태를 고쳤다. “sleep-time compute 은유망한가” 는 **문헌 전체에 대해서는 형태를 갖추지 못한 질문**이다 — 주어가 통과군과 불충족군을 섞고, 통과군 안에서도 경로마다 비용 구조가 다르다. 질문이 답할 수 있는 형태가 되려면 **경로를 지정하고, 같은자를 세우고, 판정 불가 칸은 판정 불가로 남기는** 세 단계가 선행해야 한다.

첫 단계는 이 권이 끝냈다. 다음 권들은 통과군 9 편이 실제로 무엇을하는지를 경로별로 읽고, 그 뒤에 비용과 용량을 같은 축에 올린다. 그때비로소 “얼마나 벌었는가” 를 물을 수 있다.